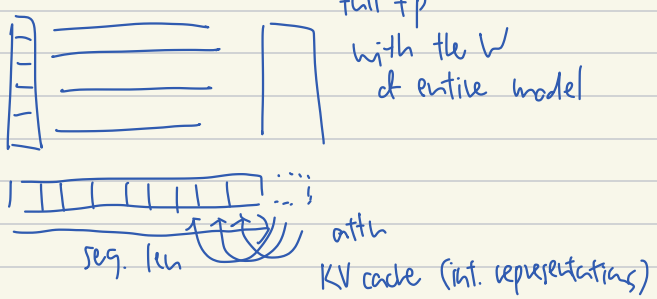


How batch size affects token cost and speed

24/9/2016

Big effect	Batch size
2 principles of analysis	Roofline analysis mem. bw cap. perf. Weights KV cache $t \geq \max(t_{mem}, t_{compute})$ $t_{compute} = \text{mhl all active params}$ $\star$ caveat: didn't count attention $\frac{B \cdot N_{active}}{FLOPs}$ $t_{mem} \geq \frac{N_{total} + B \cdot \overset{\text{spare attn}}{\text{len ctx}} \cdot \text{Bytes/token}}{\text{memory bytes/s}}$
N is params	
autoreg. inf.	
	full fp with the W of entire model  seq. len attn KV cache (int. representations)

Use attention to look back at prev. tokens thru KV cache

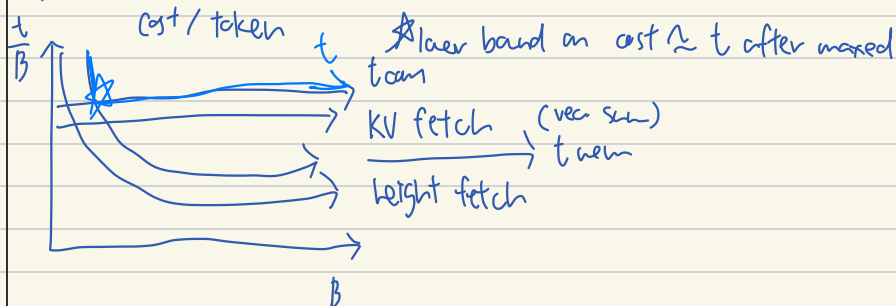
KV cache $\propto B \cdot \text{ctx len}$

t
 20ms
 t_{kv}
 KV fetch
 t_{weight}
 weight fetch
 t_{total}
 N_{total}
 B
 sum = t_{kv}
 increases w/ batch size
 see how different properties change from the latency plot as B increases

☆ same
inter. point

sensitivity to B, let ctx

KV fetch varies w/ context len
particular len. both cam/ben bound $\approx$ ideal



$nhw.$ = model N
 $den.$ = hardware N

$$\frac{B \cdot N_{\text{active}}}{\text{FLOPs}} = \frac{N_{\text{total}}}{\text{new bytes/s}}$$

fp 4

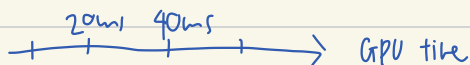
$$B \geq 300 \times \overbrace{\text{sparsity}}^8$$

$$\text{eqn:1} \quad \leftarrow \quad \frac{\text{FLOPs}}{\text{len}_{BV}} = \beta \cdot \frac{N_{\text{active}}}{N_{\text{total}}} \approx 300$$

 x^2 or x^3

optimal B size should $\uparrow$

amortization = cost of loading weights spread across generating a lot of tokens
 $\uparrow$ reduce cost/token



worst latency is 40ms

B depends on sparsity

ideal B size ≈ 2000

think of it as
users
or tokens
per second

$$\text{tok/s} = B/t = 15\text{ms} = 64B \sim 128\text{k tok/s}$$

more sparsity less compute

batch size $\uparrow$ compute bound

$$\frac{E}{K} \downarrow = \text{sparsity} \uparrow$$

fewer active params. Model performance $\downarrow$

lots of users, get a very sparse model

we want

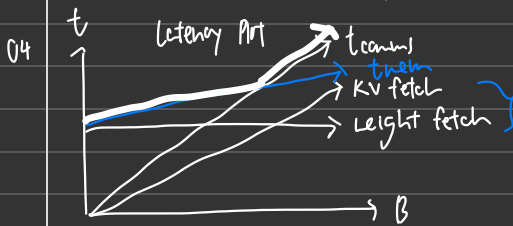
$t_{\text{com}} \downarrow$ sparsity $\uparrow$ $B \uparrow$ mem capacity $\uparrow$

How batch size affects token cost and speed

01 $t \geq (t_{mem}, t_{compute})$

02 $t_{compute} = \frac{B \cdot N_{active}}{FLOPs}$

03 $t_{mem} \geq \frac{N_{total} + B \cdot ctx_{len} \cdot KV \text{ Bytes token}}{\text{memory bw}}$



05 lower bound = $t \geq (t_{mem}, t_{compute})$ bcz even if you spent very little time on compute by processing less sequence, you still need to load the weights.

06 $t_{compute}, KV \text{ fetch}$

07 ≈ 300

08
$$\frac{B \cdot N_{\text{active}}}{\text{FLOPs}} = \frac{N_{\text{total}}}{\text{mem BW}}$$

$$B = \frac{\text{FLOPs}}{\text{mem BW}} \cdot \frac{N_{\text{total}}}{N_{\text{active}}}$$
$$= 300 \cdot \text{sparsity}$$

$$B \geq 300 \cdot \text{sparsity}$$

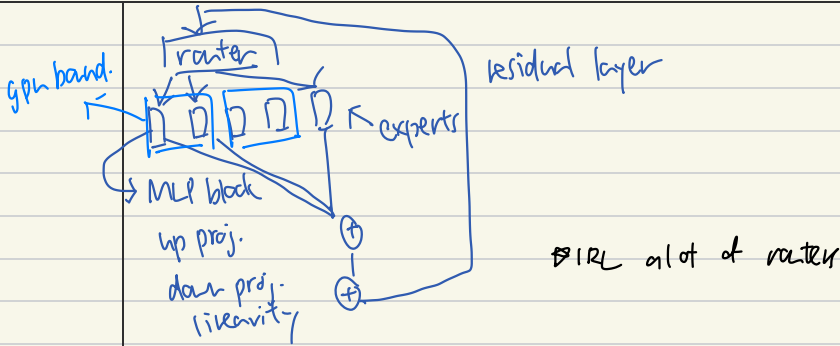
09
$$\text{HBM drain time} = \frac{\text{mem C}}{\text{mem BW}}$$

It is impossible for the HBM to load all the weights that fast

Creates idle time for FLOPs

How MoE models are laid out across GPU racks

29/9/2026



exp.
para

diff. exp. on diff. gpus

DeepSeek

256 exp.

7K 64 gpus

all to all traffic pattern
rack to rack communication

Rack

phy. structure

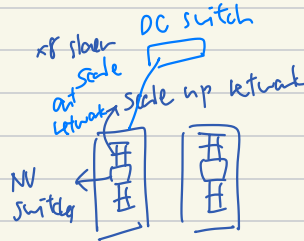
few meters tall

stores see GPUS / XPU's

pw. deli way / cooling ability

∴ 64

DC got lots of racks



hopper to BW
trays to racks

talk to other GPUS in 2 hops

$\frac{1}{2}$ tokens scale up

$\frac{1}{2}$ tokens scale out



2x racks

2x density of cables

opt.
space
weight of racks
power
cooling



pushing
physical limits

How MoE models are laid out across GPU racks

- 07 MoE communication is All-to-All traffic pattern. Within a rack, NV switches use Scale-up which is fast and perfect for All-to-All. However, between racks you connect them by datacenter switch which slows comms by $\sim 8\times$. This makes a rack a natural boundary for an MoE layer.

How pipeline parallelism spreads model layers across racks

1/10/2026

common pattern

MoE $\rightarrow$ All to All (scale up)

exp. parallelism $\rightarrow$ few. full connectivity 1 rack

$$1 < \frac{\delta \times \text{faster}}{t_{\text{scale up}}} = \frac{1}{\delta} \cdot \text{activated experts (BW)}$$

$\rightarrow$ scale up time more important

data par.

pipeline par. $\rightarrow$ change rack (create at)

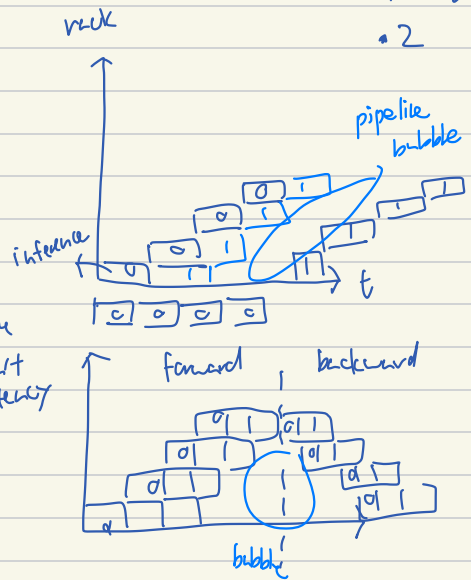
MoE layer $\uparrow$ change rack

B size into rack
no. activated GPUs

pipeline sees new capacity

$\downarrow$ do microbatches $\times$ batches

$\rightarrow$ pipeline doesn't latency



Inf. w/o pipeline

$\downarrow$ new capacity

pipeline $\times$ affect latency

How pipeline parallelism moves model layers across racks

- 01 pipeline doesn't reduce latency
- 02 You need to consolidate grad. and update the model before processing the next batch
- 03 pipelining reduces mem. capacity and KV caches obliterate memory in long context lengths, making it useless.

Why Ilya said, "As we now know, pipelining is not wise."

5/10/2016

total mem.
req.

$$C_{mem} = N_{total} + \underbrace{B \cdot len_{ctx} \cdot bytes_{tok}}_{KV's}$$

$E = \text{expert parallelism} = 4$

$P = \text{pipelining} = 4?$ $\rightarrow$ no. racks

mem
req.
/ GPU

$$c_{mem} = \frac{N_{total} + B \cdot len_{ctx} \cdot bytes_{tok}}{E \cdot P} = \frac{N_{total}}{E \cdot P} + \frac{P \cdot B \cdot len_{ctx} \cdot bytes_{tok}}{E \cdot P}$$

$$B = n_{micro} \cdot b$$

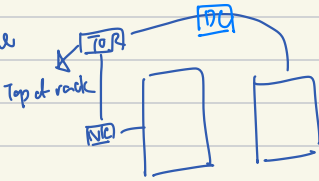
$\hookrightarrow$
 $\sim 300 \cdot \text{sparsity}$

x help
len. ctx

pipelining. useful $\rightarrow$ weights
x KV's

keep all racks usefully busy at the same time

enables
make loading
weight work
on issue



$$t_{mem, weights} = \frac{N_{total}}{\underbrace{mem\ BW}_{= Scale_{exp} \cdot GPU\ BW}}$$

$\hookrightarrow$
Solves BW problem

Why Ilya said, "As we now know, pipelining is not wise."

01 Because although it does solve the memory capacity problem, it doesn't help with latency.

Because of RL, models may be 100x over-trained beyond Chinchilla optimal

7/10/24

Chinchilla scaling

model size $\propto$ training data

best results: max model train/inf. compute

$$C_{\text{total}} = C_A + C_B$$

$\underbrace{\quad}_{\text{train}} \quad \underbrace{\quad}_{\text{inf.}}$
 $C \propto \text{tokens}$

$$C_A = \frac{1}{N} e^N$$

$$C_B = N e^{-N}$$

$$\frac{1}{10} \sim 1.5 \quad \frac{1}{2} \times 30\% = \frac{1}{10}$$

$$D_{PT} + \alpha D_{RL} = \beta \cdot D_{\text{inf}}$$

$$\downarrow$$

$$[2-6]$$

$$\downarrow$$

$$D_{PT} = \frac{2}{10} D_{RL}$$

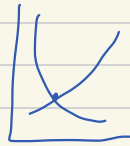
$$D_{PT} + D_{RL} = D_{\text{inf}}$$

$\downarrow$
less effi.
medium tile

$$\text{forward} = 2$$

$$\text{backward} = 2 + 2 + 2$$

$$= 6$$



$$C_{\text{train}} = N_{\text{act}} \cdot D_{PT} \cdot 6$$

$\downarrow$ ND $\uparrow$ FLOPs

$$RL = \frac{1}{N_{\text{act}}} D_{RL} \cdot \text{inefficiency}$$

$\downarrow$

$$[2-6]$$

range $\sim 30\%$

$$\text{Inf.} = 2 N_{\text{act}} D_{\text{inf}} \cdot \text{inefficiency}$$

$$D_{\text{inf}} = 500M/s \cdot 2mo$$

$$\approx 500 \times 10^6 \cdot 2 \times 30 \times 24 \times 60 \times 60$$

$$\approx 2.592 \times 10^{15}$$

$$500M/s \cdot 2mo$$

$$= 2.6 \times 10^{14} \text{ tokens}$$

$$D_{PT} \sim 150T$$

$$N_{\text{act}} \sim 100B$$

$\times 20$

$$D_{\text{chi}} \sim 2T$$

$\downarrow$
token count for PT $\rightarrow$ Chinchilla scaling law would be exceeded

$$\frac{D_{PT}}{D_{\text{chi}}} = \text{ratio of over train}$$

$$\propto N_{\text{act}} \cdot 20$$

because of RL, models may be 100x over-trained beyond GPT4-optimal

01 Forward = 2

Backward = 2+2

F+B = 6

02 $C_{total} = C_{train} + C_{RL} + C_{inf}$

$C_{train} = 6 \cdot N_{act} \cdot D_P$

$C_{RL} = [2 \cdot 6] \cdot N_{act} \cdot D_{RL} \cdot ineffic.$

$C_{inf} = 2 N_{act} D_{inf} \cdot ineffic.$

03 optimum where all three roughly equal

04 $6 \cdot D_P = 3 D_{RL} \cdot 3 \cdot ineffic. = 2 \cdot D_{inf} \cdot 3 \cdot ineffic.$

$D_P = 1.5 D_{RL} = D_{inf}$

05 $50M/s \cdot 2ms$

$= 50M/s \cdot 2 \cdot 30 \cdot 24 \cdot 60 \cdot 60$

$\approx 260T \text{ tokens}$

06 $D_{ch} \approx 20 \cdot N_{active}$

$\approx 20 \cdot 100B$

$\approx 2000B \text{ tokens}$

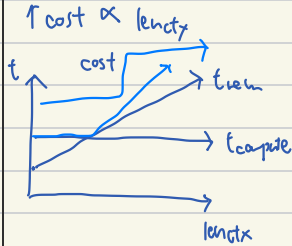
$\frac{D_P}{D_{ch}} = \text{overtrain ratio}$

$\frac{200T}{2000B} = 100\times \text{overtrained}$

Deducing long context memory costs from API pricing

10/10/26

Gemini 3.1 50% max resp. $\geq 200k$ tokens vs $\leq 200k$ tokens



so no matter what length still profitable

$$\frac{B \cdot \text{length} \cdot \text{bytes tok}}{\text{memBW}} \stackrel{!}{=} \frac{N_{\text{act}}}{\text{FLOPs}}$$

$$\text{bytes tok} = \frac{\text{memBW}}{\text{FLOPs}} \cdot \frac{N_{\text{act}}}{\text{length}}$$

$\downarrow$ $\frac{1}{300}$ $\downarrow$ $\frac{100B}{200k}$

$\approx 2KB$

dense with
over
recycle layers

$d_{\text{head}} = 128$

space with
sparsity

dense with

stored in mem

$$1 \cdot \underbrace{n \text{ attn layers}}_{\text{sparsity}} \cdot 2 \cdot \underbrace{d_{\text{head}}}_{128} \cdot \underbrace{n_{kv} \text{ heads}}_{1-8}$$

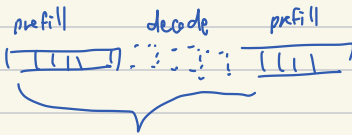
$\uparrow$ $\uparrow$
 11 $2KB$



or
asso. w/
9 head

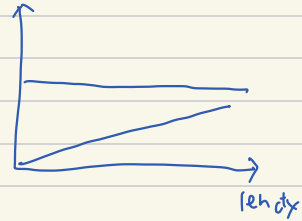
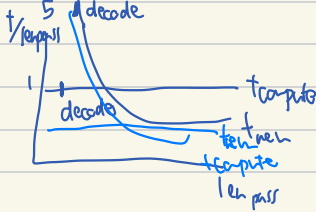
$$t_{\text{compute}} = \frac{BN_{\text{act}}}{\text{FLOPs}} \cdot \text{len pass}$$

for next tok



prefill compute lim.

decode bw lim.



\$5 ③ $\text{reformat} = \frac{N_{\text{act}}}{\text{FLOPs}} \times \text{GPU } \$/s$

↓ rebuild KV caches

Cost to move to HBM

Cost to hold on

Cost to getx
renew
compute

① store in HBM $\frac{\text{bytes tok}}{\text{HBM cap.}} \times \text{GPU } \$/s$

(20ms)

② store in DDR $(1-10s)$

Flash (1m)
HDD (1h)

$\frac{\text{bytes tok}}{\text{DDR BW}}$

$\frac{\text{storage cap}}{\text{BW}} = S_{\text{min}}$

$\frac{\text{bytes tok}}{\text{HBM cap}} \times \text{GPU } \$/s$

$\frac{\text{bytes tok}}{\text{DDR cap}} \cdot \text{DDR } \$/s$

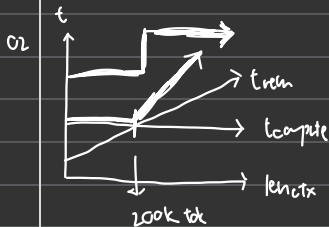
1070

$\frac{t_{\text{hold}}}{t_{\text{ret}}} = \text{fraction of cap}$

store things for so long that the time
= how long you get things out

Reducing inference memory costs from API pricing

01 Because t_{mem} scales with len_{ctx} which means costs also scale with len_{ctx} .



03

$$\frac{B \cdot len_{ctx} \cdot bytes_{tok}}{memBW} = \frac{B \cdot N_{act}}{FLOPs}$$

$$bytes_{tok} = \frac{memBW}{FLOPs} \cdot \frac{N_{act}}{len_{ctx}}$$

$$= \frac{1}{300} \cdot \frac{100b}{200k}$$

$$\approx 2KB$$

04 Model FLOPs utilization for decode is $\frac{1}{3}$ perfill. Weights loading can be amortized across many compute, but decode you're wasting FLOPs while loading weights from memory.

05 Loading KVs from memory is cheaper than recompute

Convergent evolution between neural nets and cryptography

take info that has structure vs (gradient descent)

take random extract high level info

$\frac{d}{dx}$ cipher differential cryptanalysis

Fiestal cipher/network

$f(x)$ not invertible

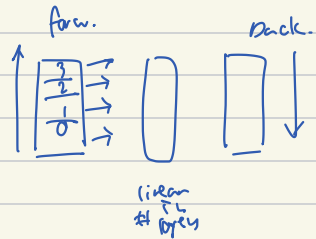
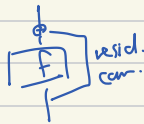
$g(x,y) = y \oplus f(x)$, x
↓
invertible

$$y = z - f(x)$$

you want layers of cipher to be invertible

Rec Nets

↓
Reversible



Convergent evolution between neural nets and cryptography

- | | |
|----|---|
| C1 | Cryptographic protocols are reversible
Neural Nets also are |
| O2 | Cryptographic protocols take in high level structured input to deduce random output
Neural Nets take in random inputs to produce high level output |